Administración de Qdrant on-premise y offline para RAG

Seguridad, diseño de colecciones e integridad (estado a Qdrant 1.19, octubre 2026)

Su intuición es correcta y está respaldada por la documentación oficial: organice la información por categoría y fuente mediante payload indexado en lugar de crear colecciones por tema; además, en su caso conviene ir un paso más allá y consolidar las colecciones “rápida” y “precisa” en una sola colección con tres named vectors (1024d, 4096d y BM25 sparse) detrás de un alias, salvo que exista un requisito de aislamiento de seguridad entre categorías.

Resumen

- **Diseño:** Qdrant recomienda explícitamente evitar una colección por tenant/tema (“Each collection carries its own resource overhead”) y usar una colección con partición por payload; como sus dos colecciones contienen los mismos chunks con modelos distintos, una única colección `kb_vN` con named vectors `fast` (1024), `precise` (4096) y `bm25` (sparse, modifier IDF) le da atomicidad por punto, un solo esquema de payload/índices y búsqueda híbrida multi-etapa en una sola llamada a la Query API.
- **Seguridad:** una instancia autoalojada “no es segura por defecto” (escucha en todas las interfaces y sin autenticación); estar offline no elimina las amenazas internas. Mínimos: API key + read-only key + JWT RBAC por colección, TLS también en LAN, no publicar puertos fuera de la red interna de Docker, imagen `-unprivileged`, `telemetry_disabled: true`, Web UI deshabilitada en producción y audit log (disponible desde v1.17).
- **Integridad:** IDs deterministas (UUIDv5) + metadatos de procedencia en el payload, upserts con `wait=true`, snapshots copiados a otro disco y probados con restauración real (solo restaurables en la misma versión menor o la siguiente), monitoreo vía `/metrics`, `/readyz` y estado de colección, y actualizaciones offline de una versión menor a la vez (p. ej. 1.17.x → 1.18.x → 1.19.x).

Hallazgos principales

1. **La recomendación oficial es “pocas colecciones + payload”.** La guía de multitenancy de Qdrant dice que crear una colección por tenant “is rarely the most efficient approach”, que cada colección tiene su propio overhead y que solo conviene crear varias “when you have a limited number of tenants that need strict isolation”. El Production Checklist añade que muchas colecciones generan demasiados shards y degradan el rendimiento, y recomienda “partition by payload instead of creating separate collections per user”. La documentación “Configure Clusters” de Qdrant Cloud fija el máximo en 1000 colecciones por clúster y advierte que “Larger numbers of collections lead to performance degradation”, lo que indica que el diseño esperado es de pocas colecciones.
2. **La regla práctica oficial es “una colección por modelo de embedding”… pero los named vectors permiten varios modelos en una misma colección.** Cada named vector tiene su propia dimensión, métrica, datatype, tier de memoria, cuantización e índice HNSW. Desde v1.18 se pueden añadir y eliminar named vectors en una colección existente (#8605), lo que elimina el principal argumento para separar colecciones al cambiar de modelo.

3. **El payload no es un mecanismo de autorización.** JWT RBAC aplica permisos a nivel de colección, no de filtro de payload; si ciertas fuentes/categorías deben ser invisibles para ciertos consumidores y estos acceden directamente a Qdrant, la única frontera que Qdrant hace cumplir es la colección. Si todo pasa por su servicio gRPC de confianza, este debe inyectar el filtro obligatoriamente.
4. **Los índices de payload deben crearse antes de la ingesta.** El checklist oficial lo dice literalmente: el HNSW solo queda optimizado para filtrado si se construye después de crear el índice de payload; si ya ingirió datos, la FAQ indica forzar la reconstrucción con un cambio mínimo de `m` o `ef_construct`.
5. **El vector de 4096d domina el costo de memoria.** Con la fórmula oficial (vectores × dimensión × 4 bytes × 1,5), 1 millón de chunks a 4096d en float32 son ~24,6 GB si se mantienen en RAM, frente a ~6,1 GB a 1024d. La cuantización (scalar int8 4×, binaria 1/1,5/2 bits desde v1.15, TurboQuant desde v1.18) y los tiers de memoria (v1.19) son las palancas principales.
6. **Las novedades 2025-2026 relevantes:** v1.15 (BQ 1,5/2 bits, cuantización asimétrica, abandono de RocksDB), v1.16 (tiered multitenancy, ACORN, inline storage, API de update condicional), v1.17 (audit logging, rotación de API key con `alt_api_key`, eliminación de RocksDB), v1.18 (TurboQuant, named vectors añadibles/eliminables, low memory mode, `enforce_internal_auth`, consulta `POST /audit/logs`), v1.19 (datatype Turbo4, parámetro unificado `memory` con tiers `pinned`/`cached`/`cold`, IDF por tenant para BM25, prefix match en índices keyword, slicing, cuotas de recursos y eliminación de los endpoints `/search`, `/recommend` y `/discover`).

1. Seguridad en un despliegue local/offline

Punto de partida. La documentación de seguridad es tajante: “Self-hosted open source deployments are **not** secure by default and are **not** production-ready” y, por defecto, “they are open to all network interfaces and do not have any kind of authentication configured”. Que su red no tenga salida a internet reduce la exposición externa, pero no protege frente a otros hosts de la LAN, contenedores vecinos comprometidos o credenciales filtradas.

Autenticación y autorización (escalonada):

Mecanismo	Desde	Uso recomendado en su sistema
Admin API key (`service.api_key`)	v1.2	Solo administración (creación de colecciones, aliases, snapshots); nunca en el servicio de consulta
Read-only API key (`service.read_only_api_key`)	v1.7	Servicio de recuperación del RAG (solo lectura sobre todas las colecciones)
JWT RBAC (`service.jwt_rbac: true`)	v1.9	Tokens con `exp` y `access` por colección: `rw` para el pipeline de ingesta sobre `kb_vN`, `r` para el retriever
`alt_api_key`	v1.17	Rotación de la admin key sin downtime (los JWT firmados con la clave antigua deben regenerarse)
`enforce_internal_auth`	v1.18	Solo relevante si pasa a clúster (puerto 6335)

Detalles operativos importantes: los JWT se firman con HS256 usando la admin API key, por lo que esa clave debe quedar exclusivamente en el host de administración; si cambia la `api_key`, todos los tokens existentes se invalidan; el claim `value_exists` permite revocar tokens sin rotar la clave (apuntando a un punto de una colección de usuarios). Según las notas de versión oficiales de Qdrant, los filtros de payload dentro del JWT se deprecaron en v1.15 y se eliminaron en v1.16 (#7450: las API keys que los usan son rechazadas), por lo que no deben usarse como aislamiento. La tabla de permisos oficial muestra que crear/borrar colecciones, actualizar aliases y recuperar snapshots exige nivel *manage*, mientras que un token de colección `rw` sí puede crear índices de payload y snapshots de esa colección.

TLS, incluso en la red local. La doc indica que "Connections are unencrypted by default. This allows sniffing and MitM attacks" y que la API key debe ir siempre sobre TLS. Qdrant soporta TLS nativo para REST y gRPC (`service.enable_tls`, `tls.cert`, `tls.key`), mTLS con `service.verify_https_client_certificate` + `tls.ca_cert`, y recarga de certificados con `tls.cert_ttl` (solo REST). Con una CA interna offline (p. ej. `step-ca` o una CA OpenSSL) puede emitir certificados para el servicio Qdrant y certificados de cliente para el retriever y la ingesta.

Puertos y red. REST 6333, gRPC 6334 y p2p de clúster 6335 (este último nunca protegido por API key salvo `enforce_internal_auth`; en nodo único simplemente no se expone). Recomendaciones:

- En Docker, deje `service.host: 0.0.0.0` dentro del contenedor (si fija 127.0.0.1 dentro del contenedor el port-mapping deja de funcionar) y controle la exposición en el host: o bien no publique puertos y conecte su servicio RAG por una red Docker `internal: true`, o bien publique solo en loopback/IP privada (`127.0.0.1:6334:6334`).
- Si su RAG usa solo gRPC, puede no publicar el 6333 hacia otras máquinas; mantenerlo accesible solo desde el host de administración para `/metrics`, snapshots y health checks. `grpc_port: null` deshabilita gRPC (debe ponerse `null` explícitamente; comentar la clave en un overlay no basta).
- `enable_cors: false` (por defecto está en `true`, lo que permite que cualquier navegador consulte la API REST) y `max_request_size_mb` ajustado a su tamaño de lote real.
- `enable_snapshot_url_recovery: false` mitiga SSRF al impedir recuperar snapshots desde URLs arbitrarias; además, v1.17.1 incluyó un parche de seguridad que fuerza la recuperación de snapshots solo desde el directorio de snapshots, y v1.18.1 autoriza la petición antes de aceptar la subida de un snapshot, razones adicionales para mantenerse en parches recientes.

Ejecución sin root y hardening del contenedor. La doc oficial afirma que "Qdrant does not need the privileges of the root user for any purpose" y recomienda la imagen `qdrant/qdrant:<versión>-unprivileged` (UID 1000), un sistema de archivos raíz de solo lectura y montar como escribibles solo `/qdrant/storage` y `/qdrant/snapshots`. Añada `cap_drop: [ALL]`, `no-new-privileges` y límites de memoria.

Telemetría. La imagen open source envía estadísticas de uso anónimas por defecto (el `config.yaml` de la imagen trae `telemetry_disabled: false`, y en el arranque se registra "Telemetry reporting enabled"). En un entorno offline esto genera intentos de conexión salientes inútiles y, sobre todo, contradice la política de "cero egress". Desactívela con `telemetry_disabled: true`, `QDRANT__TELEMETRY_DISABLED=true` o `--disable-telemetry`; esto no afecta a sus propios endpoints `/metrics` y `/telemetry`.

Web UI. El dashboard (`/dashboard`) es útil en desarrollo, pero en producción amplía la superficie de ataque (permite ejecutar llamadas a la API y generar JWT si se dispone de la clave). No existe un interruptor documentado en `config.yaml`; un mantenedor de Qdrant indicó en la discusión

#3901 de GitHub que puede deshabilitarse apuntando `service.static_content_dir` a una ruta inexistente, y el código fuente lee `service.enable_static_content` (por defecto `true`), por lo que `QDRANT__SERVICE__ENABLE_STATIC_CONTENT=false` también funciona. Si quiere conservarla, expóngala solo vía túnel SSH al host de administración.

Auditoría y logs. Desde v1.17 el audit log registra en JSON todas las operaciones que requieren autenticación/autorización (acción, timestamp, clave/usuario, colección, resultado e IP). No está habilitado por defecto, se rota diaria u horariamente con `max_log_files` (7 por defecto) y puede crecer rápidamente. Desde v1.18 acepta IDs de trazabilidad (`x-request-id`, `traceparent`) y se consulta vía `POST /audit/logs` (nivel manage). Propague el request ID de su servicio gRPC para correlacionar consultas del RAG con entradas de auditoría, y envíe los ficheros a su SIEM/almacenamiento local inmutable.

Ejemplo de configuración (`config/production.yaml`):

```
log_level: INFO
telemetry_disabled: true

service:
  host: 0.0.0.0            # dentro del contenedor; la exposición se controla en Docker
  http_port: 6333
  grpc_port: 6334
  enable_tls: true
  verify_https_client_certificate: true   # mTLS con certificados de su CA interna
  enable_cors: false
  max_request_size_mb: 64
  jwt_rbac: true
  enable_static_content: false            # deshabilita la Web UI
  enable_snapshot_url_recovery: false
  # api_key y read_only_api_key se inyectan por variables de entorno/secrets:
  # QDRANT__SERVICE__API_KEY, QDRANT__SERVICE__READ_ONLY_API_KEY

tls:
  cert: /qdrant/tls/cert.pem
  key: /qdrant/tls/key.pem
  ca_cert: /qdrant/tls/ca.pem
  cert_ttl: 3600

storage:
  storage_path: /qdrant/storage
  snapshots_path: /qdrant/snapshots       # montar en un disco distinto

audit:
  enabled: true
  dir: /qdrant/storage/audit
  rotation: daily
  max_log_files: 30
  trust_forwarded_headers: false
```

Ejemplo de `docker-compose.yml`:

```
services:
  qdrant:
    image: qdrant/qdrant:v1.19.2-unprivileged@sha256:<digest-verificado>
    user: "1000:1000"
    read_only: true
    cap_drop: [ALL]
    security_opt: ["no-new-privileges:true"]
    environment:
      QDRANT__SERVICE__API_KEY_FILE: ""   # ver nota: usar secrets/env inyectado por su orquestador
      QDRANT__TELEMETRY_DISABLED: "true"
    volumes:
      - ./config/production.yaml:/qdrant/config/production.yaml:ro
      - ./tls:/qdrant/tls:ro
      - /srv/qdrant/storage:/qdrant/storage
```

```
      - /mnt/backup-disk/qdrant-snapshots:/qdrant/snapshots
    networks: [rag_internal]
    deploy:
      resources:
        limits:
          memory: 48g
networks:
  rag_internal:
    internal: true    # sin salida y sin puertos publicados; el retriever se une a esta red
```

Nota: Qdrant lee la API key de `service.api_key` o `QDRANT__SERVICE__API_KEY`; no dé por hecho que existe una variante `_FILE`. Inyecte el valor desde su gestor de secretos en el arranque y evite dejarlo en el repositorio o en el `production.yaml`.

Amenazas relevantes aun estando offline:

- **Acceso interno no autorizado / movimiento lateral:** cualquier host o contenedor que alcance el 6333/6334 sin autenticación puede leer, sobrescribir o borrar colecciones. Mitigación: red interna, API keys, mTLS, JWT con mínimo privilegio.
- **Contenedor comprometido (p. ej. el de vLLM o el de ingesta):** que el retriever use solo una clave/JWT de lectura limita el radio de impacto; el servicio de ingesta es el único con `rw`, y solo sobre su colección.
- **Prompt injection indirecta y envenenamiento vía documentos indexados:** OWASP lo cataloga como LLM08:2025 “Vector and Embedding Weaknesses” (envenenamiento, fuga entre contextos, inversión de embeddings). Mitigación: validar y sanear documentos antes de indexarlos (texto oculto, instrucciones embebidas), guardar en el payload `trust_level`/`source` y filtrar por ellos en la recuperación, y tratar el contexto recuperado como dato no confiable en el prompt de vLLM.
- **Fuga de datos:** los snapshots contienen todo (vectores y payload en claro) y los embeddings permiten reconstruir parcialmente el texto; cifre el disco de snapshots, restrinja quién puede descargarlos (incluso la read-only key puede *descargar* snapshots según la tabla de permisos) y no guarde PII innecesaria en el payload.

2. Administración de colecciones

Por qué no crear colecciones por tema. Cada colección implica sus propios segmentos, shards, WAL, optimizadores e índices HNSW; con muchas colecciones de tamaños dispares se multiplican la memoria base, los hilos de optimización y la complejidad operativa (backups, aliases, permisos), y además se pierde la posibilidad de buscar transversalmente entre temas en una sola consulta. Con partición por payload, un filtro `category = X` sobre un índice keyword le da el mismo resultado lógico con un solo grafo y un solo ciclo de vida.

Cuándo SÍ separar colecciones:

- Requisito de **aislamiento de seguridad** que Qdrant deba hacer cumplir (JWT por colección) frente a consumidores que acceden directamente.
- **Ciclos de vida** realmente distintos (p. ej. un corpus efímero que se reconstruye a diario frente a otro estable) o requisitos de backup/retención diferentes.
- **Configuraciones incompatibles a nivel de colección** (sharding, replicación, parámetros de optimizador) o cargas que requieren hardware distinto.
- **Migraciones de modelo** cuando no se usan named vectors (patrón azul-verde con alias).

Distinta dimensión o métrica, por sí solas, ya **no** justifican separar: los named vectors admiten dimensiones y métricas distintas por vector.

Índices de payload. Tipos: `keyword`, `integer` (con opciones `lookup`/`range`), `float`, `bool`, `datetime`, `uuid`, `geo` y `text` (full-text). Buenas prácticas:

- Indexe solo campos usados en filtros, y créelos **antes** de la ingesta.
- `is_tenant: true` (desde v1.11) en el campo de partición principal, el que aparece en casi todas las consultas: co-localiza en disco los puntos del mismo valor y permite lecturas secuenciales. Es una pista de almacenamiento/rendimiento, no un control de acceso.
- `payload_m` construye subgrafos HNSW por valor de payload; con `m: 0` + `payload_m: 16` se elimina el grafo global y se dejan solo subgrafos por tenant, lo que solo tiene sentido si **nunca** busca sin ese filtro (en RAG transversal normalmente no conviene).
- ACORN (v1.16) mejora la calidad de búsquedas con varios filtros de selectividad débil, explorando vecinos de vecinos cuando los directos quedan filtrados.
- Prefix match en índices keyword (v1.19, `"prefix": true`) es útil para filtrar por rutas o URLs de fuente (`source_uri` que empieza por `\\fileserver\legal\`).
- Tiered multitenancy (v1.16) y custom sharding están pensados para muchos tenants con tamaños muy desiguales; en un nodo único con decenas de categorías no los necesita.

Named vectors para su caso (1024d + 4096d + BM25):

Aspecto	**Dos colecciones (actual)**	**Una colección con named vectors**
Consistencia entre “rápida” y “precisa”	Debe sincronizarse en la aplicación; riesgo de divergencia	Un upsert escribe los tres vectores y el payload del mismo punto
Payload e índices	Duplicados y propensos a divergir	Un único esquema
Búsqueda multi-etapa	Dos llamadas y fusión en cliente	Una llamada a la Query API con `prefetch` anidado
Memoria	Payload e ID tracker duplicados	Payload único; cada vector conserva su propio HNSW y tier de memoria
Cambio de modelo	Nueva colección + alias	Añadir/eliminar un named vector (v1.18+) o colección nueva + alias
Aislamiento de seguridad entre vías	Posible por colección	No aplica (misma colección)
Tamaño de snapshot / restauración	Dos ficheros más pequeños	Un fichero más grande

Contras a vigilar: un snapshot más grande y un solo punto de fallo lógico; si alguna vez quiere que la vía “rápida” contenga un subconjunto distinto de documentos, deberá modelarlo con filtros (los puntos pueden omitir algún named vector, pero eso complica la consistencia).

Búsqueda híbrida y reranking con la Query API (v1.10+). Patrón recomendado para su vía “precisa”: recuperación amplia con 1024d + BM25, fusión RRF, y reordenación con el vector de 4096d, todo en una llamada:

```
from qdrant_client import QdrantClient, models

flt = models.Filter(must=[models.FieldCondition(key="category",
match=models.MatchValue(value="legal"))])

res = client.query_points(
```

```
    collection_name="kb",                      # alias
    prefetch=[
        models.Prefetch(
            prefetch=[
                models.Prefetch(query=q_fast_1024, using="fast", filter=flt, limit=100),
                models.Prefetch(query=models.SparseVector(indices=q_idx, values=q_val),
                                using="bm25", filter=flt, limit=100),
            ],
            query=models.FusionQuery(fusion=models.Fusion.RRF),
            limit=50,
        )
    ],
    query=q_precise_4096,                      # rescoring con Qwen3-Embedding-8B
    using="precise",
    query_filter=flt,
    limit=10,
    with_payload=True,
)
```

La vía “rápida” sigue siendo una consulta simple con `using="fast"`. DBSF es la alternativa a RRF basada en normalización de scores; compare ambas con un conjunto de consultas etiquetadas. Si utiliza `precise` **solo** para rescoring, puede desactivar su HNSW (`m: 0`), como recomienda la documentación de consultas híbridas para vectores usados únicamente en reordenación, con un gran ahorro de RAM y tiempo de indexación; si también quiere búsquedas directas sobre 4096d, mantenga su HNSW.

Sparse BM25. Declare `modifier: idf` para que Qdrant calcule el IDF sobre el corpus; en ese caso su codificador debe enviar solo los pesos de frecuencia de término (TF normalizado por longitud), sin aplicar IDF otra vez. Use el mismo tokenizador/vocabulario en ingesta y consulta (versiónelo en el payload). Con una sola colección multitema, el IDF se calcula sobre todo el corpus del shard; la documentación advierte que la partición por payload no aísla las estadísticas IDF, y v1.19 añadió IDF por tenant para restringir el corpus del IDF a un subconjunto. Si genera BM25 en el servidor (`Qdrant/bm25`), v1.19 permite desactivar stemming y stopwords en inglés para un pipeline neutral al idioma, más adecuado para contenido en español o técnico.

Aliases y versionado. La aplicación siempre consulta `kb` (alias), nunca `kb_v3`. Para un cambio de modelo o de chunker: cree `kb_v4`, reingeste en segundo plano, valide con consultas de referencia y cambie el alias de forma atómica; el rollback es el cambio inverso. Desde v1.18 existe además la alternativa sin segunda colección: añadir el named vector nuevo, rellenarlo con `UpdateVectors` y eliminar el antiguo cuando termine; la API de update condicional de v1.16 ayuda a no pisar puntos durante esas migraciones.

```
client.update_collection_aliases(change_aliases_operations=[
    models.DeleteAliasOperation(delete_alias=models.DeleteAlias(alias_name="kb")),
    models.CreateAliasOperation(create_alias=models.CreateAlias(collection_name="kb_v4",
alias_name="kb")),
])
```

Optimización para 4096 dimensiones.

- **Estimación de RAM** (fórmula oficial de capacity planning): `vectores × dimensión × bytes_por_dim × 1,5`. Para 1 millón de chunks: `precise` float32 ≈ 24,6 GB; `fast` float32 ≈ 6,1 GB. La documentación actual también desglosa el HNSW por vector (`base × m × 2 × 4 bytes × 1,2`) y pide sumar cada named vector por separado.
- **Datatype:** `float16` reduce a la mitad (2 bytes/dim); `uint8` es para vectores ya cuantizados; `turbo4` (v1.19) guarda solo 4 bits por coordenada; el blog de v1.19 cifra la reducción en nueve veces (de 36 bits por coordenada, float32 original más copia de 4 bits, a 4 bits), pero sin poder hacer rescoring con originales.

- **Cuantización:** scalar int8 (4×, opción segura por defecto con `rescore` y oversampling); binaria 1 bit (32×), 1,5 bits (~21× según la documentación actual de cuantización; el blog de lanzamiento de v1.15 hablaba de 24×) y 2 bits (16×) desde v1.15, más asimétrica (vectores binarios, consulta en 8 bits); TurboQuant (v1.18; anunciado como compresión 8× "without the recall tax", 4 bits por defecto = 8×). La binaria funciona mejor con dimensiones altas y distribuciones centradas en cero, por lo que es candidata natural para 4096d, pero el checklist oficial insiste en medir el recall antes de producción.
- **Tiers de memoria (v1.19):** `memory: pinned | cached | cold` sustituye a `on_disk`, `always_ram` y `on_disk_payload` (siguen funcionando pero deprecados). Patrón típico: originales de 4096d en `cold`, vectores cuantizados y HNSW en `pinned`, payload en `cold`.
- **HNSW:** `m` 16–32 y `ef_construct` 100–256 como punto de partida; `ef` en consulta según el recall medido. Para 4096d, `m` más alto encarece mucho la construcción.
- **Protección de recursos:** cuotas de memoria y disco (v1.19, sustituyen a `max_resident_memory_percent` de v1.18) y `low_memory_mode` (v1.18) para evitar OOM al arrancar.

```
client.create_collection(
    collection_name="kb_v1",
    vectors_config={
        "fast": models.VectorParams(size=1024, distance=models.Distance.COSINE,
                                    hnsw_config=models.HnswConfigDiff(m=16, ef_construct=128)),
        "precise": models.VectorParams(
            size=4096, distance=models.Distance.COSINE,
            datatype=models.Datatype.FLOAT16,
            on_disk=True,   # en v1.19+: memory=models.Memory.COLD
            quantization_config=models.ScalarQuantization(
                scalar=models.ScalarQuantizationConfig(type=models.ScalarType.INT8,
                                                       quantile=0.99, always_ram=True)),
            hnsw_config=models.HnswConfigDiff(m=16, ef_construct=128),  # m=0 si solo hace rescoring
        ),
    },
    sparse_vectors_config={"bm25": models.SparseVectorParams(modifier=models.Modifier.IDF)},
    on_disk_payload=True,   # en v1.19+: payload=models.PayloadStorageParams(memory=models.Memory.COLD)
)
```

3. Integridad de la información

IDs deterministas e idempotencia. Genere el ID del punto con UUIDv5 sobre una clave estable, por ejemplo `uuid5(NAMESPACE, f"{source_id}:{doc_version}:{chunk_index}")`. Así un reintento de ingesta sobrescribe en lugar de duplicar. Alternativa: incluir el hash del contenido del chunk, que deduplica contenido idéntico entre fuentes pero obliga a borrar explícitamente los chunks antiguos cuando un documento cambia. El patrón más robusto para actualizar un documento es: (1) upsert de los chunks nuevos con `doc_version` incrementada, (2) `delete` por filtro `doc_id = X AND doc_version < N`, ambos con `wait=true`.

Esquema de payload propuesto:

Campo	Tipo de índice	Propósito
`category`	keyword, `is_tenant: true` (si casi todas las consultas filtran por categoría)	Partición principal
`source`	keyword	Sistema/origen (SharePoint, wiki, PDF…)

Campo	Tipo de índice	Propósito
`source_uri`	keyword con `prefix: true` (v1.19)	Ruta/URL original, filtrado por prefijo
`doc_id`	keyword	Borrado/reemplazo por documento
`doc_version`	integer (`range`)	Versionado del documento
`content_hash`	keyword	Deduplicación y verificación (SHA-256 del texto del chunk)
`chunk_index`	integer	Reconstrucción de contexto vecino
`lang`	keyword	Filtro de idioma
`access_level` / `trust_level`	keyword	Filtrado de seguridad aplicado por el gateway y defensa ante envenenamiento
`ingested_at` / `doc_date`	datetime	Frescura y auditoría
`embed_fast_model`, `embed_precise_model`, `sparse_encoder`, `chunker_version`	keyword (indexar solo si filtra)	Procedencia; detectar puntos pendientes de reindexar
`ingest_run_id`	keyword	Trazabilidad y rollback de una ejecución
`text`	sin índice (o `text` si usa full-text)	Contenido del chunk para el prompt

Consistencia entre "rápida" y "precisa". Con una colección y named vectors, la consistencia por punto la garantiza Qdrant: el upsert incluye los tres vectores. Si mantiene dos colecciones, aplique: los mismos IDs deterministas en ambas; un manifiesto de ingesta (tabla en su base de datos relacional o fichero) con `doc_id`, `doc_version` y estado por colección; escritura en ambas con `wait=true` y reintento idempotente; y un job de reconciliación periódico que compare recuentos con la API `count` filtrando por `doc_id`/`ingest_run_id` y repare las diferencias. El slicing de v1.19 permite repartir un `scroll` completo entre workers paralelos para esa reconciliación.

Durabilidad. Qdrant escribe cada operación en el WAL antes de aplicarla; `wait=true` hace que la respuesta llegue cuando el cambio ya está aplicado (en lugar de solo recibido), lo que es imprescindible para pipelines que leen inmediatamente lo escrito o borran después de insertar. En un nodo único, `write_consistency_factor`, `replication_factor` y `ordering` no aportan redundancia; su única protección frente a la pérdida del disco son los snapshots copiados fuera.

Snapshots y backups offline.

- **Snapshot de colección** (`POST /collections/{name}/snapshots`): granular y restaurable vía API (`PUT .../snapshots/recover` desde fichero local o subida). No incluye los aliases; recréelos tras restaurar.
- **Snapshot completo** (`POST /snapshots`): incluye todas las colecciones y aliases; solo para despliegues de nodo único y se restaura al arrancar con `./qdrant --storage-snapshot <fichero>`.
- **Compatibilidad:** un snapshot solo puede restaurarse en la misma versión menor (con parche igual o superior) o en la siguiente menor; por ejemplo, uno de v1.18.1 sirve en 1.18.x (x ≥ 1) o en 1.19.x. Registre la versión exacta junto a cada backup.
- **Espacio:** según la guía oficial "Migration and Recovery", la restauración necesita unas dos veces el tamaño de la colección en disco, porque el fichero del snapshot y la colección

restaurada coexisten hasta que termina. Qdrant no tiene retención nativa de snapshots; elimínelos con un cron tras copiarlos.

- **Verificación:** calcule y guarde el SHA-256 de cada snapshot, cópielo a un disco/NAS distinto (por defecto `snapshots_path` está en el mismo volumen que los datos) y haga simulacros de restauración periódicos en una instancia aislada, comprobando el recuento de puntos y un conjunto de consultas de referencia. Hay incidencias abiertas de fallos de recuperación (p. ej. el issue #10273 en v1.17.0), lo que refuerza que un backup no está validado hasta que se restaura.

```
# Snapshot de colección (con la admin key o un JWT rw de esa colección)
curl -s --cacert ca.pem -X POST "https://qdrant.internal:6333/collections/kb_v1/snapshots?wait=true" \
     -H "api-key: $QDRANT_ADMIN_KEY"
# Snapshot completo (colecciones + aliases)
curl -s --cacert ca.pem -X POST "https://qdrant.internal:6333/snapshots" -H "api-key:
$QDRANT_ADMIN_KEY"
# Restauración de un snapshot completo (nodo único, al arrancar)
./qdrant --storage-snapshot /qdrant/snapshots/full-snapshot-2026-10-21.snapshot
```

Monitoreo.

- `/metrics` (formato Prometheus/OpenMetrics, desde v1.17 puede servirse en un puerto HTTP dedicado); añada `?per_collection=true` para desglosar latencias por colección.
- `/healthz`, `/livez`, `/readyz` (v1.5): use `/readyz` para la disponibilidad real, porque tras un reinicio la reproducción del WAL en colecciones grandes puede tardar.
- `/telemetry` (con `details_level`) y la información de colección. Estados: **green** (lista), **yellow** (optimizando), **grey** (optimización pendiente tras reinicio; se reactiva con cualquier actualización o con “Trigger Optimizers” en la UI) y **red** (error, requiere intervención).
- Slow request log y desglose de memoria por componente (v1.18) para detectar qué parte (HNSW, cuantizados, payload) consume RAM.
- Alertas mínimas: estado distinto de green durante más de N minutos, uso de disco > 70 % (WAL, snapshots y optimizaciones necesitan holgura), latencia p95 de `query`, errores 4xx/5xx por colección y número de puntos por colección frente al manifiesto de ingesta.

Actualizaciones de versión offline.

1. En una máquina con conexión: `docker pull qdrant/qdrant:v1.19.2-unprivileged`, anote el digest, `docker save` a un `.tar`, calcule el SHA-256 y trasládelo por el canal aprobado; en destino, `docker load` y verificación del digest. Fije siempre versión y digest, nunca `latest`.
2. Actualice **una versión menor a la vez**, pasando por el último parche de cada versión intermedia (la guía oficial: de 1.15 a 1.17 hay que pasar por 1.16.3). Esto es obligatorio también en nodo único; la transición de RocksDB a Gridstore hace imposible saltar de 1.15.x a 1.17.x.
3. Antes de cada salto: snapshot completo + snapshots de colección, lectura de las notas de versión y actualización de clientes. Ojo a los cambios incompatibles: v1.17 cambió el formato de los vectores en las respuestas gRPC (actualice `qdrant-client`), y v1.19 eliminó `/search`, `/recommend` y `/discover` en favor de `/query`. Mantenga el SDK dentro de una versión menor del servidor; actualice primero el servidor.
4. Tras actualizar: verifique `/readyz`, que todas las colecciones vuelven a green, los recuentos y las consultas de referencia.

Recomendaciones

Decisión de arquitectura.

1. **Confirmado:** mantenga pocas colecciones y modele categoría, fuente, versión y procedencia en el payload con índices creados antes de la ingesta.
2. **Consolide** “rápida” y “precisa” en `kb_v1` con named vectors `fast` (1024, Qwen3-Embedding-0.6B), `precise` (4096, Qwen3-Embedding-8B) y `bm25` (sparse, `modifier: idf`), detrás del alias `kb`. Ganará consistencia atómica, un solo esquema y la consulta multi-etapa en una sola llamada. Mantenga dos colecciones solo si necesita que Qdrant aísle el acceso entre ellas o si sus ciclos de reindexación son realmente independientes.
3. **Cree colecciones adicionales** solo para: (a) corpus con restricción de acceso que deba aplicar Qdrant mediante JWT por colección; (b) migraciones azul-verde (`kb_v2`, `kb_v3`…); (c) corpus con ciclo de vida radicalmente distinto.
4. **Para la vía precisa:** empiece con scalar int8 + `float16` + originales en disco, mida el recall@10 frente a float32 en su conjunto de evaluación y solo después pruebe binaria 2 bits/asimétrica o TurboQuant. Valore desactivar el HNSW de `precise` si solo la usa para rescoring.

Checklist operativa

Seguridad

- [ ] Imagen `vX.Y.Z-unprivileged` fijada por digest; `read_only`, `cap_drop: ALL`, `no-new-privileges`, límite de memoria.
- [ ] Sin puertos publicados fuera de la red interna de Docker (o solo en loopback/IP privada); `enable_cors: false`.
- [ ] `api_key` solo en el host de administración; `read_only_api_key` o JWT `r` para el retriever; JWT `rw` con `exp` para la ingesta; rotación con `alt_api_key`.
- [ ] TLS (idealmente mTLS) con CA interna, incluido el canal gRPC.
- [ ] `telemetry_disabled: true`; Web UI deshabilitada en producción; `enable_snapshot_url_recovery: false`.
- [ ] Audit log activo, correlación con request IDs y envío a almacenamiento inmutable.
- [ ] Saneamiento de documentos antes de indexar y `trust_level`/`access_level` aplicados por el gateway.

Backups

- [ ] Snapshot completo diario + snapshots de colección antes de cada reindexación o actualización.
- [ ] Copia a un disco distinto, SHA-256 registrado, cifrado en reposo y retención por cron.

- [ ] Simulacro de restauración mensual en instancia aislada, con recuentos y consultas de referencia.
- [ ] Versión de Qdrant anotada junto a cada snapshot (compatibilidad: misma menor o siguiente).

Monitoreo

- [ ] Prometheus scrapeando `/metrics` (`?per_collection=true`), sonda `/readyz`, alertas de estado de colección ≠ green, disco > 70 %, latencia p95 y recuento de puntos frente al manifiesto.

Reindexación

- [ ] IDs UUIDv5 deterministas, upserts y deletes con `wait=true`, manifiesto de ingesta con `ingest_run_id`.
- [ ] Cambios de modelo/chunker: nueva colección + alias, o named vector nuevo (v1.18+) con backfill; evaluación antes del cambio; rollback por alias.
- [ ] Actualizaciones de Qdrant offline, una versión menor a la vez, con snapshot previo y clientes actualizados.

Advertencias

- Varias afirmaciones sobre funciones recientes (TurboQuant “sin recall tax”, compresión de Turbo4, ACORN) provienen de anuncios y benchmarks del propio fabricante; mídalas con sus datos y sus embeddings de Qwen3 antes de adoptarlas.
- El método para deshabilitar la Web UI (`enable_static_content` / `static_content_dir`) no figura en la referencia oficial de configuración; procede del código fuente y de una respuesta de un mantenedor, por lo que podría cambiar entre versiones. Verifíquelo tras cada actualización (por ejemplo, que `/dashboard` devuelva 404).
- La eliminación de los filtros de payload en JWT está confirmada por las notas de versión oficiales (deprecados en v1.15 y eliminados en v1.16, #7450); la documentación actual solo describe permisos por colección, así que revise cualquier token antiguo que los incluya.
- Las cifras de RAM son estimaciones de orden de magnitud con la regla del 1,5×; el consumo real depende de `m`, del número de segmentos, del payload y del tier de memoria. Use el desglose de memoria de v1.18+ para medir.
- La versión más reciente observada es v1.19.x (etiquetas `v1.19.2-unprivileged` en Docker Hub; chart de Helm 1.19.1 de septiembre de 2026). Revise las notas de versión vigentes antes de actualizar, porque Qdrant publica versiones con frecuencia.

Fuentes principales

- Qdrant — Multitenancy: https://qdrant.tech/documentation/manage-data/multitenancy/

- Qdrant — Production Checklist: https://qdrant.tech/documentation/production-checklist/
- Qdrant — Capacity Planning: https://qdrant.tech/documentation/capacity-planning/
- Qdrant — FAQ: https://qdrant.tech/documentation/faq/qdrant-fundamentals/
- Qdrant — Security: https://qdrant.tech/documentation/security/
- Qdrant — Configuration: https://qdrant.tech/documentation/ops-configuration/configuration/
- Qdrant — Usage statistics: https://qdrant.tech/documentation/ops-configuration/usage-statistics/
- Qdrant — Collections: https://qdrant.tech/documentation/manage-data/collections/
- Qdrant — Hybrid search: https://qdrant.tech/articles/hybrid-search/
- Qdrant — Snapshots: https://qdrant.tech/documentation/snapshots/
- Qdrant — Monitoring: https://qdrant.tech/documentation/ops-monitoring/monitoring/
- Qdrant — Upgrades: https://qdrant.tech/documentation/upgrades/
- Qdrant — Configure Clusters (Cloud): https://qdrant.tech/documentation/cloud/configure-cluster/
- Qdrant blog — 1.16.x: https://qdrant.tech/blog/qdrant-1.16.x/ · 1.19.x: https://qdrant.tech/blog/qdrant-1.19.x/
- GitHub releases — v1.15.0, v1.17.0, v1.18.0, v1.18.1: https://github.com/qdrant/qdrant/releases
- GitHub — config.yaml: https://github.com/qdrant/qdrant/blob/master/config/config.yaml
- GitHub — discusión #3901 (Web UI): https://github.com/orgs/qdrant/discussions/3901
- GitHub — issue #10273: https://github.com/qdrant/qdrant/issues/10273
- Docker Hub — qdrant/qdrant tags: https://hub.docker.com/r/qdrant/qdrant/tags
- OWASP / CrossClassify — Seguridad en RAG: https://www.crossclassify.com/resources/articles/owasp/securing-rag-applications-against-poisoned-knowledge-cross-user-data-leakage-and-manipulated-retrieval/